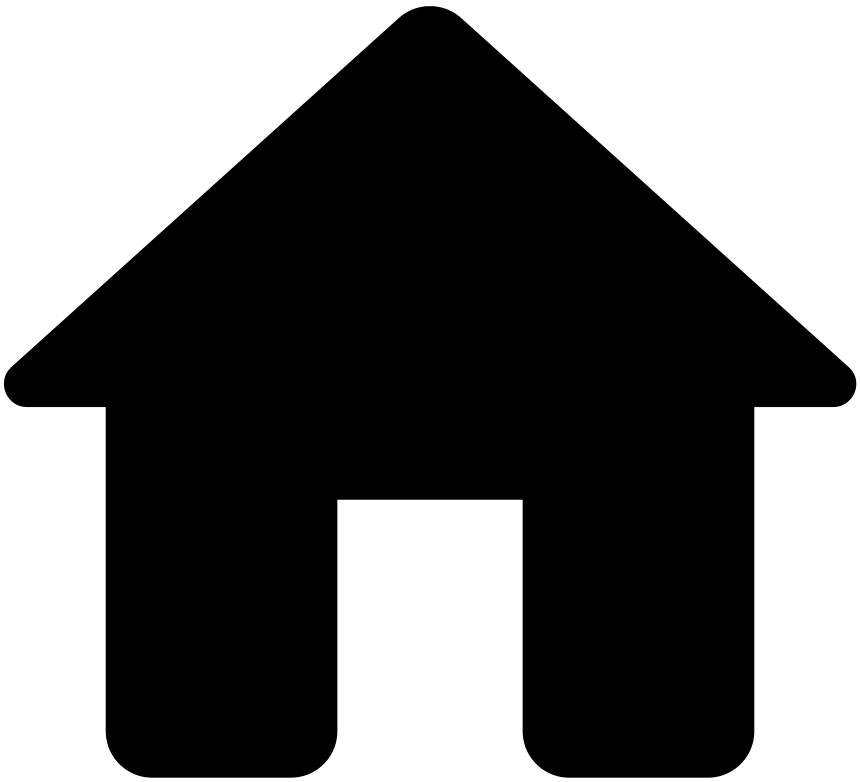


Metrics Best Practices

Metrics can be seen as the "basic unit" of Data Science work. Use cases like fraud prevention and anti-money laundering revolve around the concept of **change in behavior**. This is where metrics shine, by providing a straightforward way of aggregating data points through time efficiently. Metrics are also commonly referred to as **profiles** or **historical profiles**.

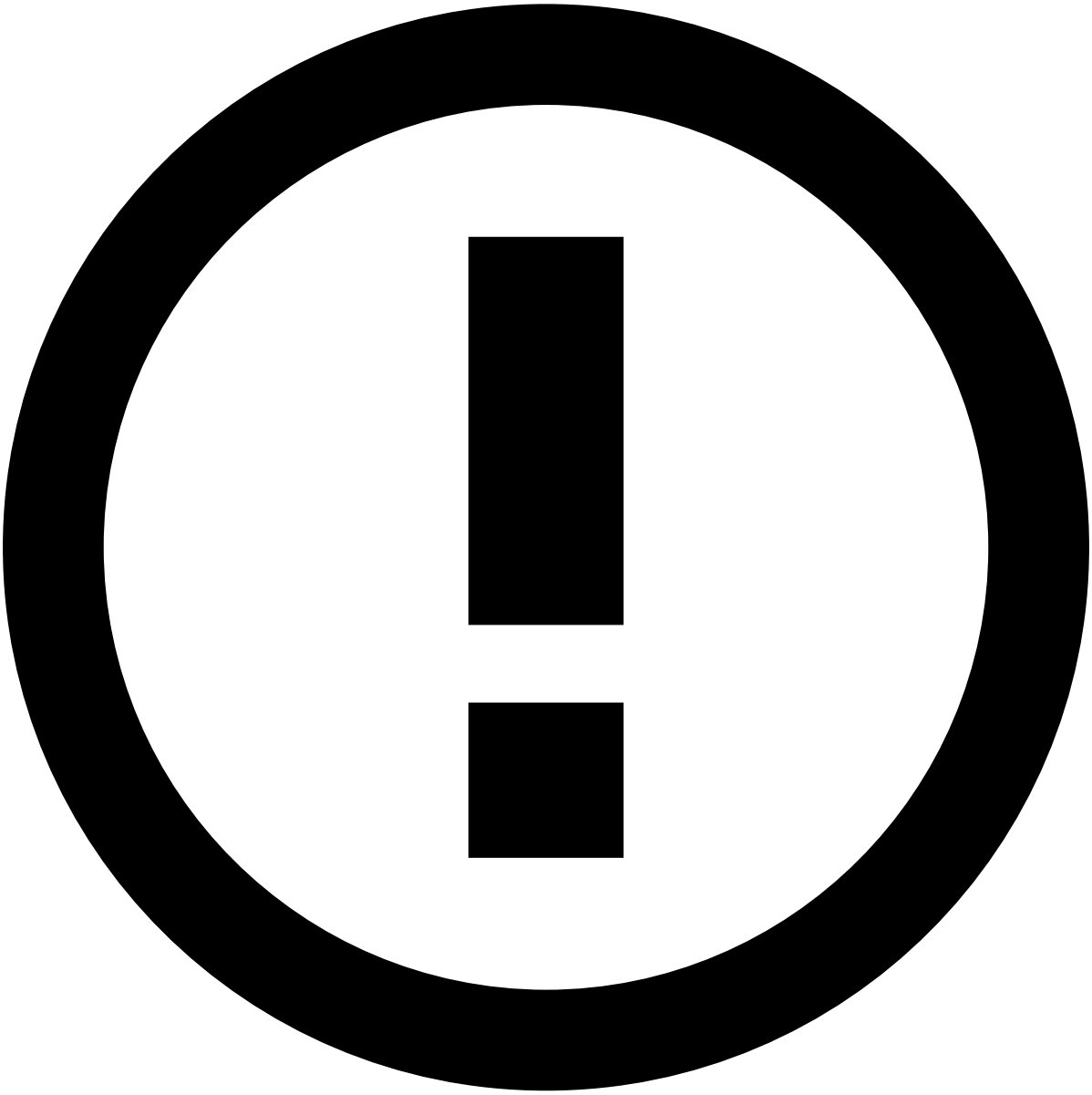
This chapter will cover topics such as the differences between Data Science Platform and Runtime, and a few optimizations that are helpful to bear in mind. In addition to the general best practices section below, the chapter includes subpages covering different aspects of best practices for working with metrics.

When thinking of a typology for features, we can conceive the following categories:

- **Atomic**: features that rely only on information present in their respective instance. Example: "Keyboard distance of the email address".

- **Composite**: Features that combine two or more previously computed values. Example: "Ratio between transaction amount and account balance".
- **Lookup**: Features that depend on a lookup table. This table can have the following behaviors:
    - **[Near-]static**: Never or rarely updated. Example: mapping country codes to coordinates, MCC to MCG.
    - **Dynamic**: Updated in batches at regular but sporadic intervals. Example: risky MCC, risky countries, etc.
- **Metrics**: Temporal aggregations. These features require the definition of a time window, an aggregation field, and how the metric should update. Metrics can optionally include filters. Example: average money transferred by this customer in the last 24 hours.

Generally, you should only create features through the metrics operator if they fall into the "Metrics" category explained above.



info

Metrics are the most complex computations that the system will do both offline and in runtime. Expressions in metrics are written in Pulse Query Language (PQL). For details on semantics, symbols, and operators see the official documentation.

As a rule of thumb, when defining the **Window update period**, it is recommended that short windows (1 day or less) be set to **real time**, i.e. events enter the window as they arrive; whereas longer windows (1 week, 1 month, ...) should be **scheduled**, i.e. events enter the window in batches. This aspect of how a window is updated is one of the major constraints when designing metrics. It will be explained more in depth in the Constraints page.

General Good Practicesâ□□

The following are some general good practices to consider when designing your metrics.

Dosâ□□

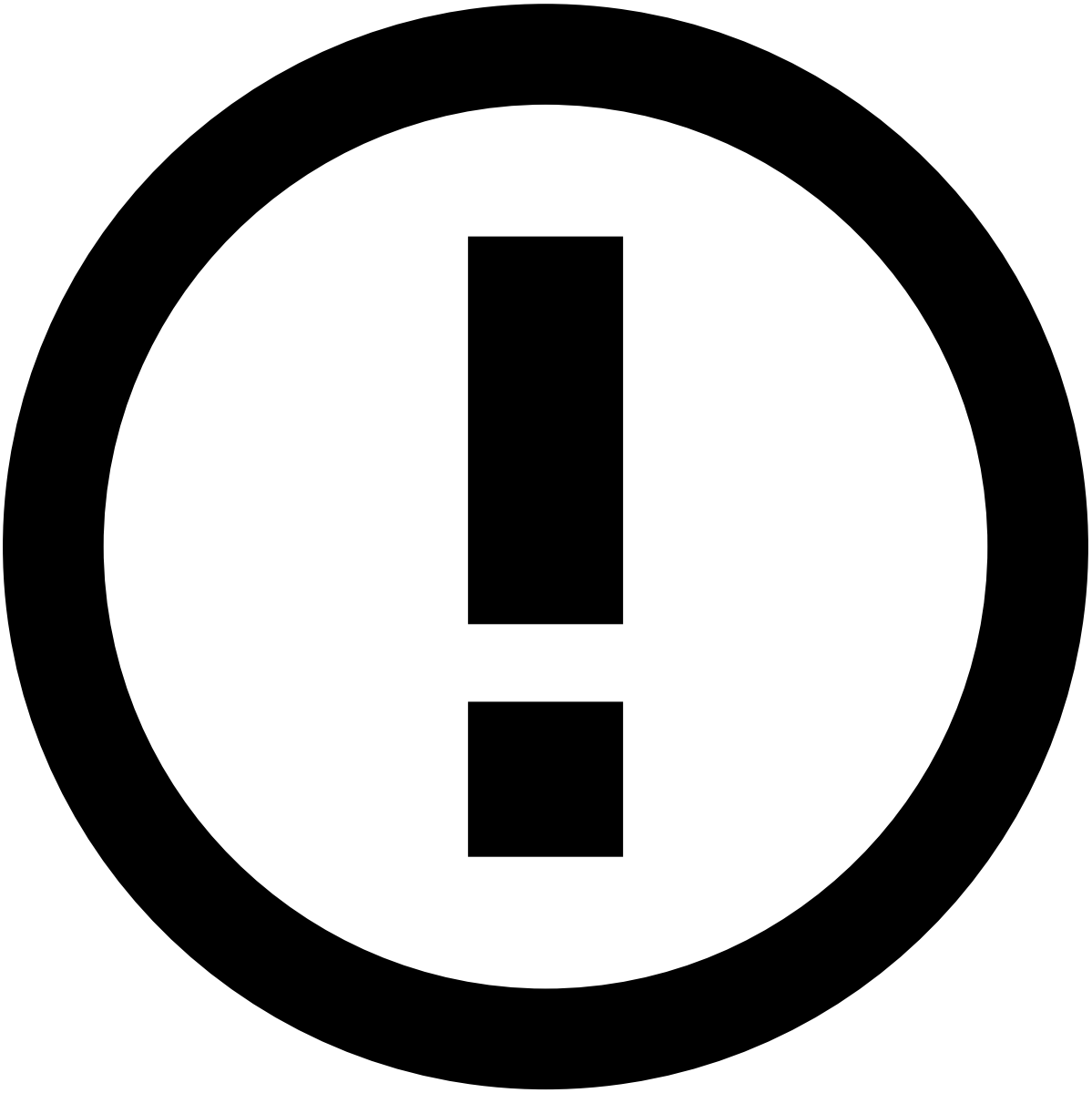
- Metrics in the same Metrics operator cannot depend on each other.
- Best: 1 big metrics box with all the metrics in it (SMJs or not).
- Second best: 1 box with SMJs and another with real-time metrics.
- Make sure to **enable *Output incomplete instances*** in the metrics operators when running jobs in the Data Science Platform. Otherwise, the plan output will be missing data corresponding to the largest profile window (for example, if the dataset goes from Jan to Dec and the largest window is 3 months, the output dataset will start in April). This is especially important when there are metrics with very large windows (which could result in an empty output dataset) or multiple metrics operators (the effect will compound, and overall the sum of the largest window of each operator will be cut from the output).

Don'tsâ□□

- Avoid dependencies between metrics.
- Avoid cascading Scheduled Metrics Jobs (SMJs).
- **Avoid redundant metrics**. While you may be tempted to compute the count of transactions for each customer over lots of different time windows (5 minutes, 15 minutes, 30 minutes, 1 hour, 3 hours, ...), it might cause a degradation in runtime performance (latency and memory) and sometimes even in DS metrics. It's a good idea to check for overly correlated features at least once during the modeling process, either directly by computing the pairwise correlations between features, or indirectly by trying different combinations of features and comparing the results of Feature Importance. For very large data sets you can compute the cross correlations in a sample of your data. Another source of excessive correlation is profiles with correlated group by fields: for example, card number and customer ID likely overlap a lot, while card number and merchant ID less so.

Advanced techniquesâ□□

- **Salting grouping entities with lots of missing values**. Suppose you want to compute profiles grouped by the field `ip_address`. However, in your dataset about half of the transactions don't have an IP address. You start by adding a filter to the metric window, something like `ip_address != null`; however this is not enough: when computing the profile, all the transactions will still be loaded and distributed to the executors before the filter is applied. Thus, there will be a very large partition with all the missing IPs, which will likely crash the Spark job. One way to circumvent that is to substitute that missing value by something else, such as the transaction ID. For that, create a new field `ip_address_salted` in a map operator before the metrics operator. Assign it a value similar to `ip_address ?? transaction_id`. Then, update the metric window filter to `ip_address_salted != transaction_id`. This will still yield the same end result, but without generating a massive partition in the Spark job.
- It's seldom necessary to use cascading metrics (a metric that depends on a different metric from a previous metrics operator). However, if it is a deal-breaker for the project (such as in compliance cases) it is important to notify the team responsible for runtime deployments, as there is an important property that needs to be set to ensure correct computation of cascading metrics.



Advanced notes

Metrics in plans within the Data Science Platform will run the Feedzai stream processing engine on top of Apache Spark. This combination of technologies allows the processing of metrics in Data Science to be horizontally scalable, while maintaining the same feature definition that goes into runtime.

Despite Spark helping on the parallelization of metrics computation, you should bear in mind "the 4 Ss": **Spill**, **Skew**, **Shuffle**, **Small Files** â□□which are the most common sources of issues when running Spark. These topics will be covered throughout this documentation, when pertinent.